Qualité du code et tests

Écrire du code qu'un autre peut lire, présenter un projet avec un README, et prouver qu'il fonctionne avec des tests automatiques (`assert` , puis `pytest`).
Ce sont les trois choses qu'un recruteur regarde en premier sur ton GitHub.

1. Le code est lu plus qu'il n'est écrit
2. Les règles de style (PEP 8)
3. Des noms explicites
4. Des fonctions courtes et documentées
5. Des commentaires utiles
6. Le README
7. Tester avec assert
8. Tester avec pytest
9. Quels cas tester ?
10. Exercices corrigés
11. À savoir par cœur

---

1. Le code est lu plus qu'il n'est écrit

> **Principe 1.1**
>
> Un code est écrit une fois, mais lu des dizaines de fois : par toi dans trois mois, par tes collègues, par un recruteur. La lisibilité n'est pas un luxe : elle réduit les bugs et le temps de correction.

```
# ✗ Illisible
def f(l):
    a=0
    for x in l:
        if x>=10:a=a+1
    return a

# ✓ Lisible : même comportement
def compter_admis(notes):
    """Renvoie le nombre de notes supérieures ou égales à 10."""
    admis = 0
    for note in notes:
        if note >= 10:
            admis = admis + 1
    return admis
```

2. Les règles de style (PEP 8)

> **Définition 2.1**
>
> **PEP 8** est le guide de style officiel de Python. Tout le monde le suit, donc tous les codes Python se ressemblent et se lisent vite.

RÈGLE	✅	❌
Indentation de 4 espaces	`return x`	2 espaces, ou des tabulations mélangées
Espaces autour des opérateurs	`total = a + b`	`total=a+b`
Variables et fonctions en `snake_case`	`note_finale`, `calculer_moyenne`	`NoteFinale`, `calculerMoyenne`
Constantes en MAJUSCULES	`NOTE_MIN = 10`	`notemin = 10`
Deux lignes vides entre deux fonctions	✓	fonctions collées
Imports en haut du fichier	`import json` en ligne 1	des imports au milieu du code

3. Des noms explicites

> **Méthode 3.1**
>
> Une **variable** porte un **nom** qui dit ce qu'elle contient : `notes_valides`, `chemin_fichier`. Une **fonction** porte un **verbe** qui dit ce qu'elle fait : `lire_notes`, `calculer_moyenne`. Un **booléen** se lit comme une question : `est_admis`, `a_des_erreurs`.

> **Remarque 3.2**
>
> Les noms d'une seule lettre sont acceptés seulement dans des boucles très courtes (`for i in range(3)`). `x`, `data`, `tmp` ou `truc` ne disent rien.

4. Des fonctions courtes et documentées

> **Méthode 4.1** *(une fonction = une tâche)*
>
> Si tu dois écrire « et » pour décrire ce que fait une fonction (« elle lit le fichier **et** calcule la moyenne **et** affiche »), c'est qu'il faut la découper. Une fonction tient généralement en moins de 20 lignes.

> **Définition 4.2** *(docstring)*
>
> Une **docstring** est un texte entre triples guillemets, placé juste sous `def` . Elle dit ce que la fonction reçoit et ce qu'elle renvoie.

```
def calculer_moyenne(notes):
    """Renvoie la moyenne de la liste notes.
    Lève ZeroDivisionError si la liste est vide."""
    return sum(notes) / len(notes)
```

5. Des commentaires utiles

> **Principe 5.1**
>
> Un bon commentaire explique **pourquoi**, jamais **quoi**. Le « quoi », c'est le code lui-même qui doit le dire, grâce à de bons noms.

```
admis = admis + 1        # ✗ ajoute 1 à admis  (inutile : le code le dit déjà)

# ✓ On recherche le mot entier : sinon « ph » serait trouvé dans « physique »
motif = r"\b" + re.escape(mot) + r"\b"
```

6. Le README

Définition 6.1

Le `README.md` est la page d'accueil d'un dépôt GitHub. Il est écrit en **Markdown** : `#` pour un titre, `-` pour une liste, des blocs de code entre trois accents graves.

Méthode 6.2 *(les 6 sections d'un bon README)*

```
# Nom du projet
Une phrase : ce que fait le projet et pour qui.

## Fonctionnement       ← les grandes étapes, en 3 à 5 points
## Installation        ← les commandes exactes (venv, pip install -r requi
## Utilisation         ← la commande pour lancer, et un exemple de résulta
## Tests               ← python -m pytest -v
## Limites             ← ce que le projet ne fait pas, honnêtement
```

7. Tester avec `assert`

Définition 7.1 *(test)*

Un **test** appelle une fonction avec un exemple dont on connaît **déjà** la bonne réponse, et vérifie que la fonction donne cette réponse.

Définition 7.2 *(assert)*

`assert condition` ne fait rien si la condition est vraie, et lève une `AssertionError` si elle est fausse. On peut ajouter un message : `assert x == 3, "x devrait valoir 3"`.

```
assert compter_admis([12, 8, 15]) == 2
assert compter_admis([]) == 0
```

Corollaire 7.3

Seule une fonction qui fait `return` peut être testée ainsi. Une fonction qui fait seulement `print` renvoie `None` : on ne peut rien comparer.

---

8. Tester avec `pytest`

Définition 8.1

pytest est l'outil de test standard en Python. Il trouve automatiquement les fichiers `test_*.py` et les fonctions `test_*`, les exécute toutes et affiche un rapport : PASSED ou FAILED.

```
# tests/test_stats.py
import pytest
from stats import compter_admis, calculer_moyenne


def test_compter_admis_cas_normal():
    assert compter_admis([12, 8, 15]) == 2


def test_compter_admis_liste_vide():
    assert compter_admis([]) == 0


def test_moyenne_liste_vide_leve_une_erreur():
    with pytest.raises(ZeroDivisionError):      # on VÉRIFIE que l'erreur arrive
        calculer_moyenne([])
```

```
python -m pip install pytest
python -m pytest -v            # -v : une ligne par test
```

Remarque 8.2

Si le code est dans `src/` et les tests dans `tests/`, un fichier `pytest.ini` à la racine, contenant `[pytest]` puis `pythonpath = src`, permet aux tests de trouver le code.

9. Quels cas tester ?

TYPE DE CAS	QUESTION	EXEMPLE POUR COMPTER_ADMIS
Normal	Ça marche dans le cas courant ?	`[12, 8, 15]` → 2
Limite	Et aux bords ?	`[10]` → 1 (10 compte bien)
Vide	Et sans données ?	`[]` → 0
Doublons	Et avec des répétitions ?	`[15, 15]` → 2
Invalide	Et avec une mauvaise donnée ?	une erreur claire, vérifiée avec `pytest.raises`
Régression	Un ancien bug peut-il revenir ?	`count_keyword("physique ph", "ph") == 1`

> **Dans le métier**
>
> En ML, on teste les fonctions de préparation des données, parce qu'une erreur à cet endroit ne fait rien planter : elle fausse silencieusement le modèle. Le badge « tests passing » sur un dépôt GitHub montre au recruteur que tu travailles comme un ingénieur.

10. Exercices corrigés

Exercice 1 · Renommer ★

```
def f(a, b):
    c = a / b * 100
    return c
```

Corrigé.

```
def calculer_pourcentage(note, bareme):
    """Renvoie la note convertie sur 100."""
    return note / bareme * 100
```

Exercice 2 · Commentaire utile ou inutile ? ★

« `# boucle sur les notes` » au-dessus de `for note in notes:`, et « `# 10 est inclus : la règle du bac est ≥ 10` » au-dessus de `if note >= 10:`.

Corrigé. Le premier est inutile : il répète le code. Le second est utile : il explique **pourquoi** on a choisi `>=`.

Exercice 3 · Écrire 4 tests pour est_admis ★★

```
def test_admis_cas_normal():
    assert est_admis(14) is True

def test_ajourne_cas_normal():
    assert est_admis(7) is False

def test_limite_10_admis():
    assert est_admis(10) is True

def test_juste_en_dessous():
    assert est_admis(9.99) is False
```

Exercice 4 · Trouver le bug grâce au test ★★★

```
def compter_admis(notes):
    admis = 0
    for note in notes:
        if note > 10:
            admis = admis + 1
    return admis

assert compter_admis([10, 12]) == 2     # AssertionError !
```

Corrigé. `note > 10` exclut le 10. Il faut `note >= 10`. C'est exactement le rôle d'un test de **cas limite**.

11. À savoir par cœur

1. Le code est lu bien plus souvent qu'il n'est écrit.
2. PEP 8 : 4 espaces, espaces autour des opérateurs, `snake_case`, CONSTANTES, 2 lignes vides entre les fonctions.
3. Variable = un nom. Fonction = un verbe. Booléen = une question.
4. Une fonction = une tâche, avec une docstring qui dit ce qu'elle reçoit et ce qu'elle renvoie.
5. Un commentaire explique le pourquoi, jamais le quoi.
6. README : description, fonctionnement, installation, utilisation, tests, limites.
7. `assert condition` : rien si c'est vrai, `AssertionError` si c'est faux.
8. pytest : des fichiers `test_*.py`, des fonctions `test_*`, lancés avec `python -m pytest -v`. `pytest.raises` vérifie qu'une erreur arrive.
9. À tester : le cas normal, les limites, le vide, les doublons, l'invalide, et les anciens bugs.
